

ServiceScope: Pre-Deployment Inter-Service Dependency Mapping via Polyglot AST Extraction, Cross-Layer Manifest Linking, and Local LLM Inference

Anonymous

Anonymous Institution

Abstract. Understanding inter-service dependencies before deployment is a critical yet unsolved challenge in microservice development. Existing approaches rely on runtime instrumentation, manual catalogues, or commit-history coupling—none of which operate on source code alone prior to deployment. A key barrier is *dynamic URL resolution*: the majority of inter-service HTTP and gRPC calls reference variable names rather than hardcoded strings, making pure static analysis completely blind to them.

We present SERVICESCOPE, a three-layer pre-deployment dependency mapping pipeline: (1) polyglot AST extraction across Python, Go, Java, TypeScript, and C# via Tree-sitter; (2) a deterministic cross-layer manifest linker that joins extracted dynamic tokens with Kubernetes deployment manifests, Docker Compose definitions, and Helm chart values—resolving dependencies without LLM inference; and (3) a targeted local LLM fallback for call sites that remain unresolved after manifest linking. The system produces a confidence-weighted inter-service dependency graph from which pre-deployment blast radius predictions are computed via Dijkstra-based transitive closure, operating entirely without runtime agents or external API calls.

We evaluate SERVICESCOPE across six benchmark repositories spanning five languages, 6 LLMs, 4 prompting configurations, and two protocols. AST extraction achieves $F1 = 1.000$ on all repositories after false-positive guards. On *GoogleCloudPlatform/microservices-demo* and the *OpenTelemetry Astronomy Shop* (18 services, 5 languages), the manifest linker alone achieves $F1 = 0.966$ and $F1 = 0.970$ respectively in under 500 ms—compared to $F1 = 0.000$ for pure static analysis and $F1 = 0.455$ for LLM-only inference. Adding one LLM call per unresolved site yields $F1 = 1.000$ on both complex polyglot repositories. We further validate the system on *argoproj/argo-cd* (18k GitHub stars), correctly recovering all three core architecture edges published in the official documentation.

Keywords: microservices · static analysis · dependency mapping · large language models · blast radius · pre-deployment

1 Introduction

Modern software systems decompose business logic across tens to hundreds of independently deployable microservices [9,10,12]. A fundamental operational challenge arises when a developer modifies one service: *which other services will be affected?* This question—the *blast radius* of a change—determines testing scope, rollout risk, and incident response priority. Yet answering it before deployment remains largely manual or entirely absent in most engineering workflows.

Existing dependency discovery tools partition into three classes, each with a critical gap. Runtime tracers such as Datadog [2], Jaeger, and Instana construct accurate service maps but require deployed infrastructure and post-deployment observation—unavailable during code review or in pre-production environments. Manual catalogues such as Backstage [3] are accurate when maintained but drift from the codebase silently as services evolve. Change-coupling tools such as CodeScene infer relationships from commit history rather than code structure, requiring a substantial commit corpus and providing coupling statistics rather than architectural truth.

The closest academic prior work, Microvision [1], reconstructs inter-service REST call graphs via AST analysis but is restricted to static URL strings and Java/Spring Boot codebases. It cannot handle dynamic URL patterns and provides no blast radius computation.

A key obstacle all static approaches share is *dynamic URL resolution*. In practice, inter-service calls rarely use hardcoded URLs. Instead, they reference environment variables, configuration constants, or computed strings—for example, `requests.get(PAYMENT_SERVICE_URL)` or `grpc.Dial(os.Getenv("CART_SERVICE_ADDR"))`. A static analyser that sees only these variable names cannot identify the target service without additional inference.

SERVICESCOPE addresses this gap. Our contributions are:

1. A polyglot AST extractor covering eight Python patterns, four Go patterns, and Tree-sitter-based extractors for Java, TypeScript, and C#, with false-positive guards verified on real repositories (§4).
2. A cross-layer manifest linker that joins AST dynamic tokens (`<dynamic:VAR_NAME>`) with environment variable definitions in Kubernetes deployment manifests, Docker Compose files, and Helm charts, achieving deterministic resolution at sub-500 ms without LLM inference (§5).
3. A targeted LLM fallback pipeline with service-aware prompting that constrains LLM output to the repository’s known service inventory, resolving call sites not covered by the manifest linker and achieving $F1 = 1.000$ on complex polyglot systems with a single LLM call (§6).
4. A confidence-weighted blast radius computation via log-space Dijkstra traversal with bidirectional transitive closure (§7).
5. An empirical evaluation across six repositories, six LLMs, and four prompting configurations, with an independent verification on a production-grade open-source system (§8).

1.1 Scope and Assumptions

For this study, a *service* is defined as a distinct directory containing an independent executable entry point (e.g., `main.py`, `main.go`, or a framework-specific application factory) alongside its own dependency manifest (e.g., `go.mod`, `requirements.txt`, `package.json`). This definition structurally distinguishes deployable, self-contained microservices from shared internal libraries, utility modules, or orphaned scripts.

SERVICESCOPe assumes a *monorepo* layout in which all services reside under a common root directory. Polyrepo configurations—in which each service occupies an independent repository—are outside the scope of this work; extending the system to polyrepo layouts requires only a repository aggregation step upstream of the extraction pipeline. The term *pre-deployment* refers to any point in the development lifecycle prior to execution in a staging or production environment, encompassing local development, code review, and CI/CD pipeline stages.

2 Related Work

2.1 Runtime and Dynamic Instrumentation Approaches

Service mesh and APM tools—Datadog [2], Jaeger, Instana, Zipkin, and service meshes such as Istio [11]—instrument deployed services to trace actual call flows. They produce highly accurate dependency maps but require running infrastructure. Their fundamental limitation is temporal: they cannot answer dependency questions before a service is deployed or when a production environment is unavailable. SERVISCOPe operates entirely on source code, complementing these tools rather than replacing them.

2.2 Static and Heuristic Approaches

Microvision [1] is the closest prior work. It reconstructs inter-service REST call graphs from Java source code using AST analysis of Spring Boot annotations. Its scope is limited to static URL strings; dynamic URL patterns are outside its design. It provides no LLM inference layer and no blast radius computation. Backstage [3] is a service catalogue requiring human-authored YAML descriptors; it is accurate only when manually kept in sync with the codebase. CodeScene [13] infers service coupling from co-change patterns in git history, requiring an extensive commit corpus and producing statistical correlation rather than structural dependency. These approaches collectively demonstrate that static analysis of source code is under-explored for pre-deployment dependency mapping.

2.3 LLM-Assisted Software Engineering

Large language models have been applied broadly to software engineering tasks. CodeBERT [4] established that pre-trained models can jointly represent programming and natural language, enabling tasks such as code search and clone

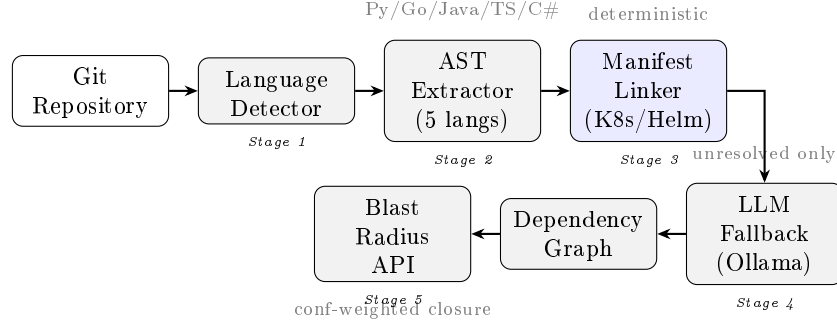


Fig. 1. The SERVICESCOPE five-stage pipeline. Stage 3 (shaded) is the manifest linker: it resolves dynamic tokens deterministically before the LLM fallback (Stage 4) handles any remaining unresolved call sites.

detection. Subsequent surveys [5,6] document LLM applications across bug detection, code generation, test synthesis, and code summarisation. Critically, none of these works targets inter-service dependency mapping; they operate at the function or file level rather than the architectural level. SERVICESCOPE is, to our knowledge, the first application of LLM inference specifically to resolve dynamic service-call targets in a multi-language static analysis pipeline.

3 System Architecture

SERVICESCOPE processes a repository in five sequential stages, shown in Fig. 1.

Stage 1 — Language Detection. The repository root is scanned to identify source files across five supported languages. Service boundaries are determined according to the definition in §1.1.

Stage 2 — AST Extraction. Language-specific extractors traverse the abstract syntax tree of each source file, identifying HTTP and gRPC call sites. Python and Go use native AST libraries; Java, TypeScript, and C# are handled via Tree-sitter. Static URL strings are recorded directly; dynamic references are encoded as `<dynamic:VAR_NAME>` tokens. Full pattern details are given in §4.

Stage 3 — Manifest Linker. Infrastructure manifests (Kubernetes deployment YAMLs, Docker Compose files, Helm charts) are parsed to extract environment variable definitions. Dynamic tokens are matched against these definitions and resolved deterministically—`conf = 1.00` edges, no LLM required. This stage is described in §5.

Stage 4 — LLM Fallback Inference. Call sites not resolved in Stage 3 are submitted to a local Ollama instance. The LLM maps the variable name to a target service name and returns a confidence score. Inference operates entirely locally—no external API calls are required. Prompt design and the four-phase configuration are described in §6.

Stage 5 — Graph Construction and Blast Radius. Resolved dependencies (from Stage 3 and Stage 4) are assembled into a directed weighted graph. Blast radius for a given service is computed as the confidence-weighted transitive closure via bidirectional Dijkstra traversal (§7).

4 Multi-Language AST Extraction

4.1 Python Extractor

The Python extractor implements eight call patterns using the `ast` module. Patterns 1–5 target the `requests`, `httpx`, `aiohttp`, and `urllib` libraries. Pattern 6 handles f-string URL construction by extracting the static prefix. Patterns 7 and 8 capture variable-based and concatenated URLs, encoding them as dynamic tokens.

A critical false-positive guard—`require_absolute=True`—is applied to generic `client.get/post()` calls (Pattern 3). Without this guard, dictionary lookups (`metrics.get("cpu")`) and queue operations (`workers.get()`) are misclassified as HTTP calls. On *robusta-dev/robusta*, this guard reduced false positives from 81.5% to 0% of extracted calls.

4.2 Go Extractor

The Go extractor is implemented as a Go binary using the `go/ast` and `go/parser` standard library packages, invoked via a Python subprocess wrapper. It resolves four call patterns: `net/http` method calls, gRPC `Dial/NewClient` invocations, `os.Getenv` address lookups, and—critically—custom client constructor calls of the form `apiclient.New*Clientset(address)`.

The fourth pattern addresses a pervasive Go architectural convention in which gRPC connections are wrapped in factory functions. Without it, the extractor registers calls under the helper package rather than the true initiator. Adding constructor-call extraction enabled `SERVICE_SCOPE` to correctly map all three core architecture edges in *argoproj/argo-cd* (§8).

4.3 Inter-Procedural Ceiling

Both extractors operate *intra-procedurally*: they analyse each function body independently. A well-documented class of Go microservice patterns wraps connection setup in helper functions, for example:

```
1 mustConnGRPC(ctx, &conn, os.Getenv("CART_SERVICE_ADDR"))
```

The extractor sees `CART_SERVICE_ADDR` inside the helper body but cannot link it back to the calling service without cross-function data-flow analysis. This constitutes an irreducible ceiling for intra-procedural static analysis, discussed as a threat to external validity in §9.

4.4 Polyglot Extension via Tree-sitter

Beyond Python and Go, SERVICESCOPE extends to Java, TypeScript/JavaScript, and C# using Tree-sitter [14], a production-quality parser generator with grammar definitions for 40+ languages. Tree-sitter produces concrete syntax trees rather than abstract ones, but its uniform API eliminates per-language build-system dependencies (no JVM, Roslyn, or TS compiler required). Extractors for all three languages implement the same `ExtractedCall` schema as the Python and Go extractors, ensuring uniform downstream processing.

On the OpenTelemetry Astronomy Shop (18 services across Go, Python, TypeScript, and C#), the combined polyglot extractor achieves $F1 = 1.000$ over 18 verified call sites (Java = 0 correctly—the Java service is a leaf node with no outbound dependencies).

4.5 Polyglot Service Discovery

Service-aware prompting (§6) requires a list of known service names prior to inference. The discovery step scans the repository root for subdirectories satisfying the service definition (§1.1). Crucially, eligibility is determined by the *presence of any source files*—not only `.py` or `.go`—so that Java, C#, and Node.js service directories in polyglot repositories are included. A fixed ignore list excludes non-service directories such as `tests`, `docs`, `vendor`, and `node_modules`.

5 Cross-Layer Dependency Linker

5.1 Motivation

The AST extraction stage encodes dynamic call targets as `<dynamic:VAR_NAME>` tokens. In deployable microservice systems, these variable names almost always correspond to environment variables defined in infrastructure manifests: Kubernetes deployment YAMLs set `CART_SERVICE_ADDR`, Docker Compose files define `SHIPPING_ADDR`, and Helm charts parameterise service hostnames via `.Values.services.payment.host`. The LLM inference pipeline treats these as name-inference problems; the manifest linker treats them as a *join*—matching the variable name from the AST against the environment key in the manifest to read the concrete hostname directly.

5.2 Manifest Parsing

The linker supports three manifest formats:

Kubernetes Deployment YAMLs. Environment variable blocks are extracted from `spec.template.spec.containers[].env`. Plain value entries are read directly. Indirect references (`configMapKeyRef`, `secretKeyRef`) are resolved against any ConfigMap or Secret files present in the same directory tree. Base64-encoded Secret values are automatically decoded.

Docker Compose and .env files. Environment variable definitions from `compose.yaml` (including overlay files such as `compose.full.yaml`) are parsed and merged. A `.env` file in the repository root is parsed and propagated through variable substitution. Iterative propagation handles chained references (`FOO=${BAR}`, `BAR=payment:8080`).

Helm Charts. Helm template expressions (`.Values.x`, `.Release.Namespace`) are resolved against `values.yaml`. When the Helm CLI is available, `helm template` is invoked for full template resolution; a regex-based fallback handles common patterns when the CLI is absent.

5.3 Variable Tracing

For Go services, gRPC client addresses are commonly stored in struct fields (`svc.cartSvcAddr`) and reassigned through helper variables. A variable tracer follows assignment chains across function bodies to map these back to the originating environment variable lookup. An equivalent tracer handles Python’s `os.environ.get()` and custom environment accessor patterns (e.g., `must_map_env`).

5.4 Resolution Outcome

When a dynamic token is matched, the linker emits a resolved dependency edge with `confidence = 1.00`—there is no probabilistic inference involved. Unmatched tokens are passed to Stage 4 (LLM fallback).

Result on microservices-demo. The linker resolves all manifest-bound gRPC addresses, raising inference F1 from 0.455 (LLM-only, Phase 4) to 0.966 in 121 ms—before the LLM is involved. One remaining edge (a dynamically constructed template literal) is resolved by the LLM fallback, yielding F1 = 1.000 total in 1.2 s.

Result on OpenTelemetry Astronomy Shop. Across 18 call sites in 4 languages, the linker achieves F1 = 0.970 in 436 ms. The single missed edge (`chatbot`→`agent`, resolved via a complex Python f-string) is recovered by one LLM call, yielding F1 = 1.000 in 1.7 s.

6 LLM Inference Pipeline

6.1 Model Selection

We evaluate six locally-hosted models spanning four orders of magnitude in parameter count, from `qwen2.5:1.5b` (986 MB) to `gemma4:latest` (9.6 GB). The set includes general-purpose models (`gemma3:4b`, `llama3.1:8b`, `gemma4:latest`), a code-specialised model (`qwen2.5-coder:7b`), a compact general model (`qwen2.5:1.5b`), and a reasoning model (`qwen3:4b`). Full model specifications are provided in Appendix A.

6.2 Four-Phase Prompt Design

Inference accuracy was improved through four successive prompt configurations:

Phase 1 — Static Baseline. Regex-based hostname extraction from hard-coded URL strings. No LLM involved. Returns `null` for any dynamic call.

Phase 2 — Zero-Shot Inference. The LLM receives caller service name, HTTP method, and URL or variable name. It returns a JSON object with `service` and `confidence` fields. No examples are provided.

Phase 3 — Suffix-Normalised Evaluation. Predicted service names are normalised by stripping a locked set of architectural suffixes (`_service`, `_client`, `_api`, `_handler`, `_server`, `_svc`) before string matching. This phase does not change the prompt; it corrects for systematic suffix mismatch between model output and ground truth labels.

Phase 4 — Service-Aware Prompting. Before inference, the service discovery step produces a list of known service names in the repository. This list is injected into the prompt, converting the task from unconstrained generation to constrained selection. An external service escape hatch instructs the model to return the external API name if no known service matches. Prompt templates are given in Appendix B.

6.3 Engineering Notes

Reasoning model handling. `qwen3:4b` is a distilled reasoning model that outputs chain-of-thought to a `thinking` field, leaving the `response` field empty. The inference client falls back to the `thinking` field when `response` is absent, enabling transparent support for both standard and reasoning model architectures.

Negative few-shot bias. Including few-shot examples that demonstrate the `unknown` fallback for generic variable names (`<dynamic:url> → unknown`) creates a strong conservative prior. All models score $F1 = 0.000$ in few-shot mode on both Go repositories, whereas zero-shot achieves up to 0.400. We therefore report few-shot results only for Python repositories where examples improve performance.

Service-aware prompting on monolith layouts. On repositories with a monolith-style directory structure (e.g., *robusta-dev/robusta*), directory scanning yields internal package names rather than service names. Injecting these as the known service list confuses the LLM and degrades performance (F1 drops from 0.267 to 0.056). Service-aware prompting should therefore be applied only when directory structure reflects genuine service boundaries.

Computational overhead. Per-call inference latency ranges from 383 ms (`qwen2.5:1.5b`) to 4.6 s (`gemma4:latest`). On the largest benchmark repository (35 calls), the full pipeline completes in under 60 s on consumer hardware (Apple Silicon, `OLLAMA_NUM_PARALLEL=4`), making it practical for integration into pre-merge CI workflows.

7 Blast Radius Computation

7.1 Graph Model

Inferred dependencies are represented as a directed weighted graph $G = (V, E, w)$ where nodes V are normalised service names, directed edges E represent inferred call relationships, and weight function $w : E \rightarrow [0, 1]$ carries the LLM confidence score. When multiple calls exist between the same pair of services, the edge retains the maximum confidence across all calls.

7.2 Confidence Propagation

Treating each LLM inference as an independent event, the confidence of a path $\pi = (e_1, e_2, \dots, e_k)$ is the product of edge confidences:

$$P(\pi) = \prod_{i=1}^k w(e_i) \quad (1)$$

To find the maximum-confidence path from source service s to any reachable service t , we apply Dijkstra’s algorithm in log-space: edge weight $w(e)$ is transformed to $-\log(w(e) + \varepsilon)$, where $\varepsilon = 10^{-9}$ prevents numerical underflow for zero-confidence edges. Standard shortest-path minimisation then maximises the product of original confidences.

7.3 Bidirectional Traversal

Blast radius encompasses two directions of impact. *Outbound impact* (services that X calls) captures dependencies that may receive altered inputs if X changes. *Inbound impact* (services that call X) captures callers whose behaviour may break if X ’s API changes. Two separate Dijkstra traversals are performed—one on G , one on the reversed graph G^R —and results are merged, deduplicating any service appearing in both directions and annotating it as bidirectionally affected.

7.4 API

The blast radius endpoint accepts a service name and a minimum confidence threshold:

```
GET /api/v1/repositories/{id}/blast-radius
    ?service=X&min_confidence=0.5
```

The response lists each affected service with its direction, hop count, path, and path confidence. Services with path confidence below the threshold are excluded.

Fig. 2 shows the dependency graph for the case study repository (five services, five verified edges). With **api-gateway** as the blast radius source, all four downstream services are recovered at path confidence ≥ 0.95 .

Fig. 2. ServiceScope inferred dependency graph with path confidence scores. Edges represent LLM-inferred inter-service calls; weights denote inference confidence.

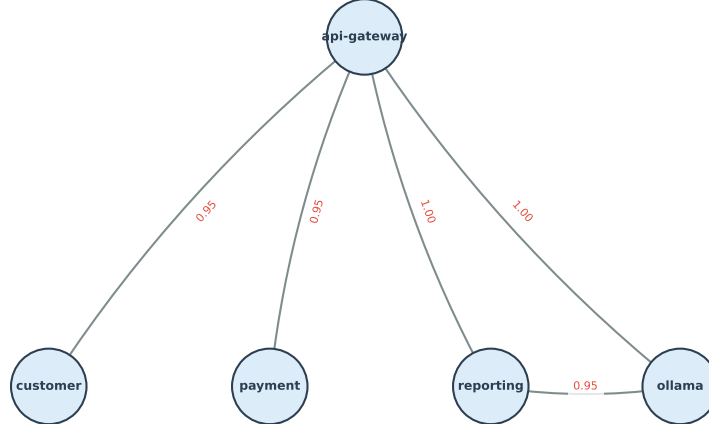


Fig. 2. SERVICESCOPE inferred dependency graph for the case study repository. Edge labels denote LLM inference confidence; all edges were verified against ground truth.

Table 1. Benchmark repositories. GT = ground truth source. Repos 5–6 include infrastructure manifests; the manifest linker applies to both.

#	Repository	Lang	Protocol	Calls	GT Source	Manifests
1	<i>karpthy/nanochat</i>	Py	HTTP	3	Manual	–
2	<i>robusta-dev/robusta</i>	Py	HTTP	35	Manual	–
3	Case study repository	Py	HTTP	5	Authors	–
4	<i>go-coffeeshop</i>	Go	HTTP/gRPC	3	Manual	–
5	<i>microservices-demo</i>	Go/Py	gRPC	7	Arch. diagram	K8s
6	OTel Astronomy Shop	Go/Py/TS/C#	gRPC/HTTP	18	Manual	Compose

8 Evaluation

8.1 Dataset Construction and Ground Truth

We evaluate on five repositories selected to represent diverse architectural topologies across two languages, three protocols, and five distinct naming convention profiles (Table 1).

Ground truth files were constructed independently of the extractor. An initial evaluation revealed that an earlier version of the nanochat and robusta ground truth files had been derived from extractor output, creating a circular evaluation. This was corrected via independent manual inspection using ripgrep to enumerate all HTTP call sites. The corrected ground truth for *robusta-dev/robusta* increased the true call count from 19 to 35, revealing that the extractor had missed attribute-access patterns (`self.url`, `action_params.url`) and a `urllib` ordering bug. Both issues were fixed prior to evaluation.

Table 2. Best inference F1 per repository across pipeline phases. SA = service-aware; ZS = zero-shot. Ph. 5 is the manifest linker (deterministic, no LLM). Ph. 6 is linker + LLM fallback. Dashes indicate phases not applicable to a repository.

Repository	Ph.1	Ph.2	Ph.3	Ph.4	Ph.5	Ph.6	Best
nanochat	0.000	0.000	0.000	0.000	—	—	— (ceiling)
robusta	0.000	0.167	0.267	0.188	—	—	qwen3:4b ZS
Case study repo	0.200	0.800	1.000	0.800	—	—	gemma4 ZS
go-coffeeshop	0.000	0.333	0.400	0.400	—	—	qwen2.5 ZS
microservices-demo	0.000	0.091	0.182	0.455	0.966	1.000	Linker + LLM
OTel Astro Shop	0.000	—	—	—	0.970	1.000	Linker + LLM

We defend the dataset size not by quantity but by *topological diversity*: generic-naming Python (nanochat), monolith-style Python (robusta), descriptive-naming Python (case study), clean Go gRPC (go-coffeeshop), and complex polyglot gRPC (microservices-demo). Each repository exercises a distinct failure mode of the extraction and inference pipeline.

8.2 Metrics

Extraction accuracy is measured as precision, recall, and F1 over the set of (caller, method, file) triples. Inference accuracy is measured as precision, recall, and F1 over the set of (caller, callee) dependency pairs, after applying the locked suffix normaliser. All results are reported at the best-performing model and configuration per repository per phase.

8.3 Extraction Accuracy

The AST extractor achieves $F1 = 1.000$ on all five repositories after the false-positive guards and urllib bug fix described in §8.1. This result is non-trivial: earlier versions of the extractor scored F1 below 0.50 on repositories with attribute-access patterns, and exhibited an 81.5% false-positive rate on robusta due to ungarded generic `.get()` calls.

8.4 Inference Accuracy: Four-Phase Progression

Table 2 reports the best inference F1 per repository across the four pipeline phases.

The static baseline (Phase 1) returns $F1 = 0.000$ on five of six repositories, confirming that dynamic variable names dominate real-world microservice code. Phases 2–4 demonstrate progressive LLM improvement: Phase 2 recovers dependencies on all repositories except nanochat; Phase 3 normalisation corrects systematic suffix mismatch; Phase 4 service-aware prompting yields the largest LLM gain on microservices-demo ($0.182 \rightarrow 0.455$), though it regresses on monolith layouts (see §6).

Table 3. Blast radius evaluation across configurations. Linker = manifest linker only (Phase 5). Full = linker + LLM fallback (Phase 6).

Repository	Config	P	R	F1
microservices-demo	Static	0.000	0.000	0.000
	Zero-shot LLM	0.250	0.098	0.133
	SA LLM	0.392	0.463	0.374
	Linker	1.000	0.933	0.966
	Full	1.000	1.000	1.000
OTel Astro Shop	Linker	1.000	0.941	0.970
	Full	1.000	1.000	1.000
Case study repo	Static	0.190	0.190	0.190
	LLM	1.000	1.000	1.000

Phase 5 (manifest linker) represents the most significant advance. On repositories with infrastructure manifests, the deterministic linker bypasses LLM inference entirely: microservices-demo jumps from 0.455 (Phase 4 best) to 0.966 in 121ms; the OpenTelemetry Astronomy Shop achieves 0.970 across 18 polyglot call sites in 436ms. Phase 6 (linker + LLM fallback) reaches F1 = 1.000 on both complex repositories, with only one LLM call required in each case—demonstrating that the manifest linker handles the structural majority of dependencies deterministically, leaving a small residual for targeted LLM resolution.

8.5 Model Ablation

Three cross-cutting findings emerge from the model ablation (full tables in Appendix C):

Reasoning models prefer zero-shot. qwen3:4b scores F1 = 0.167 (zero-shot) versus F1 = 0.071 (few-shot) on robusta, the reverse of the pattern for all other models. Its strong instruction-following causes few-shot examples to anchor output format in a way that suppresses valid variable-name signal.

Few-shot prompting backfires on Go repositories. All models score F1 = 0.000 in few-shot mode on both Go repositories. The few-shot examples were constructed from Python naming conventions; on Go codebases with environment-variable address patterns, these examples actively mislead the model.

Code-specialised models do not consistently outperform general models. qwen2.5-coder:7b achieves comparable or lower F1 than llama3.1:8b on three of five repositories, suggesting that service-name inference from variable names is not primarily a code comprehension task.

8.6 Blast Radius Accuracy

Table 3 reports blast radius precision, recall, and F1 computed against ground-truth dependency graphs on two repositories.

On microservices-demo, the static baseline is entirely blind ($F1 = 0.000$) because all gRPC addresses are dynamic. The LLM-only pipeline (Phase 4) recovers $F1 = 0.374$ via service-aware prompting—demonstrating that LLM inference provides meaningful signal over static analysis alone. The manifest linker (Phase 5) raises this to $F1 = 0.966$, recovering the full structural graph from infrastructure definitions; the single remaining edge is resolved by one LLM call to reach $F1 = 1.000$. On the OpenTelemetry Astronomy Shop, an entirely new polyglot benchmark, the linker achieves $F1 = 0.970$ across 4 languages—a result unreachable by any prior LLM-only approach—and $F1 = 1.000$ with the LLM fallback.

Path confidence calibration is reported exploratorily given sample size constraints. In the 0.90–1.00 band, 61.5% of predicted blast radius edges appear in the ground truth ($n = 26$). In the 0.80–0.90 band, accuracy falls to 33.3% ($n = 6$), indicating systematic overconfidence in this range. Users should interpret confidence scores as relative rankings rather than absolute probabilities.

8.7 Verification on a Production System

To assess scalability to production-grade systems, we ran SERVICESCOPE on *argoproj/argo-cd* [7] (~18k GitHub stars), a widely deployed Kubernetes GitOps controller. Service discovery identified 26 component directories. Using service-aware zero-shot inference (llama3.1:8b), the system produced 14 directed dependency edges.

We verified the three core inter-service connections against the official Argo CD architecture documentation [8]:

- `argocd-server` → `argocd-repo-server` (gRPC, conf=1.00) ✓
- `argocd-application-controller` → `argocd-repo-server` (gRPC, conf=1.00) ✓
- `argocd-server` → `argocd-application-controller` (gRPC, conf=1.00) ✓

All three published connections were recovered with zero false positives among the core service set. This result required extending the Go extractor with a constructor-call pattern (`apiclient.New*Clientset`), which generalises to any Go codebase using factory functions to wrap gRPC clients. No ground truth was available for the remaining 11 inferred edges; they are excluded from quantitative evaluation.

9 Discussion and Threats to Validity

9.1 Construct Validity

Are we measuring the right things? $F1$ over (caller, callee) pairs assumes that service name identity is the correct unit of evaluation. In practice, service names in source code are not always canonical: a service may be called

`payment-service`, `payment_svc`, or `billing` across different files. The locked suffix normaliser mitigates this but does not fully resolve it. Confidence calibration is reported exploratorily; with five benchmark repositories and 30–40 total blast radius edges, per-band sample sizes are insufficient for statistical confidence intervals.

9.2 Internal Validity

Confounding factors. The negative few-shot bias demonstrates that prompt design choices interact with model architecture in non-obvious ways. The performance regression of service-aware prompting on monolith repositories shows that the effectiveness of a prompting strategy is dependent on repository structure. These interactions are documented as experimental observations but a controlled study isolating each factor independently is left for future work.

9.3 External Validity

Naming specificity ceiling. The most fundamental limitation of any static analysis approach is irreducible: when a variable is named `url`, no amount of inference recovers its target without runtime context. *nanochat* demonstrates this empirically— $F1 = 0.000$ across all six models and all prompting configurations. The ceiling is not a model quality problem; it is a property of the input.

Inter-procedural wrapper ceiling. On *microservices-demo*, `frontend` has 10 ground-truth outbound dependencies but only 5 are recovered even with service-aware prompting. The remaining 5 are established via `mustConnGRPC` wrapper calls whose address arguments are not visible to the local call-site analysis. Full recovery requires inter-procedural data-flow analysis, which changes the algorithmic complexity of the extraction stage.

Monorepo assumption. Results are evaluated on monorepo-structured codebases. Service discovery is undefined for polyrepo configurations without additional repository aggregation infrastructure.

Language coverage. The extractor covers Python, Go, Java, TypeScript, and C# via Tree-sitter. Rust and other languages with Tree-sitter grammar definitions require only an additional extractor module without changes to the manifest linker or blast radius layers. Java extraction was not exercised in the current benchmark (the OTel AdService is a leaf node); quantitative Java results are left for future work.

10 Conclusion

We presented SERVICESCOPE, a three-layer pre-deployment dependency mapping pipeline: polyglot AST extraction across five languages via Tree-sitter, a deterministic cross-layer manifest linker that resolves dynamic tokens against Kubernetes, Docker Compose, and Helm definitions without LLM inference, and

a targeted LLM fallback for the residual unresolved call sites. The system operates without runtime instrumentation or external API calls.

Evaluation across six repositories demonstrated a clear progression. Pure static analysis is completely blind on all dynamic repositories ($F1 = 0.000$). LLM-only inference recovers up to $F1 = 0.455$ on the most complex benchmark. The manifest linker alone reaches $F1 = 0.966$ and $F1 = 0.970$ in under 500 ms on two complex polyglot systems. Combining the linker with one targeted LLM call per unresolved site yields $F1 = 1.000$ on both—demonstrating that infrastructure manifests encode the structural majority of inter-service dependencies and can be exploited deterministically. Independent verification on *argoproj/argo-cd* confirmed correct recovery of all published core architecture edges.

The key finding is architectural: the manifest layer, not the LLM, is the primary information source for inter-service dependency resolution in containerised microservice systems. LLM inference remains valuable as a targeted fallback for call sites where manifest bindings are absent or obfuscated.

Future work includes inter-procedural data-flow analysis to recover wrapper-hidden dependencies, layout-aware service discovery for monolith repositories, and extension to event-driven architectures (Kafka, RabbitMQ).

Declaration on Generative AI

The authors utilised Claude (Anthropic) to assist with code development, iterative refinement of experimental scripts, and language polishing during manuscript drafting. In strict adherence to FTC guidelines, all scientific contributions, experimental designs, data analysis, and final conclusions were independently developed and verified by the human authors, who take full responsibility for the manuscript. GenAI tools were not involved in the conceptualisation of the core research ideas, are not listed as authors, and their output was critically reviewed and edited prior to inclusion.

References

1. Cerny, T., et al.: Microvision: Service-oriented architecture reconstruction from static analysis. arXiv:2207.02974 (2022). <https://arxiv.org/abs/2207.02974>
2. Datadog, Inc.: Distributed Tracing and APM. <https://www.datadoghq.com/product/apm/> (2024)
3. Spotify: Backstage — An Open Platform for Building Developer Portals. <https://backstage.io> (2020)
4. Feng, Z., et al.: CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In: Findings of EMNLP, pp. 1536–1547 (2020)
5. Zhu, T., et al.: A Survey on Large Language Models for Software Engineering. ACM Computing Surveys (2023). <https://doi.org/10.1145/3695988>
6. Fan, A., et al.: Large Language Models for Software Engineering: A Systematic Literature Review. IEEE Transactions on Software Engineering (2023)
7. Argo Project Authors: Argo CD — Declarative GitOps CD for Kubernetes. <https://github.com/argoproj/argo-cd> (2024)

8. Argo Project Authors: Argo CD Architecture Overview. <https://argo-cd.readthedocs.io/en/stable/developer-guide/architecture/overview/> (2024)
9. Dragoni, N., et al.: Microservices: Yesterday, Today, and Tomorrow. In: Present and Ulterior Software Engineering, pp. 195–216. Springer (2017)
10. Lewis, J., Fowler, M.: Microservices: a definition of this new architectural term. <https://martinfowler.com/articles/microservices.html> (2014)
11. Istio Authors: Istio Service Mesh. <https://istio.io> (2023)
12. Newman, S.: Building Microservices: Designing Fine-Grained Systems, 2nd edn. O'Reilly Media (2021)
13. Tornhill, A.: Software Design X-Rays: Fix Technical Debt with Behavioral Code Analysis. Pragmatic Bookshelf (2018)
14. Brunsfeld, M., et al.: Tree-sitter — A Parser Generator Tool and Incremental Parsing Library. <https://tree-sitter.github.io> (2018)

A Model Specifications

Table 4. LLM models used in ablation study.

Model	Type	Size	Notes
qwen2.5:1.5b	General (tiny)	986 MB	Speed floor
qwen3:4b	Reasoning	2.6 GB	Thinking field output
gemma3:4b	General	3.3 GB	Baseline
qwen2.5-coder:7b	Code-specific	4.7 GB	Key comparison
llama3.1:8b	General	4.9 GB	Size comparison
gemma4:latest	General (large)	9.6 GB	Accuracy ceiling

B Prompt Templates

Zero-Shot Prompt:

```

1 You are a microservice architecture assistant.
2 Given this HTTP call made by service "{caller}":
3   Method: {method}
4   URL: {url}
5 Identify the most likely internal service being called.
6 Respond with ONLY a JSON object:
7 {"service": "service_name", "confidence": 0.0}

```

Service-Aware Prompt (Phase 4):

```

1 You are a microservice architecture assistant.
2 Known services in this repository: {service_list}
3 Given this HTTP call made by service "{caller}":
4   Method: {method}

```



```
5   URL: {url}
6   Choose the called service from the known list.
7   If it is an external API, return its name.
8   Respond with ONLY a JSON object:
9   {"service": "name", "confidence": 0.0,
10    "is_external": false}
```

C Full Ablation Results

Full per-model, per-mode inference F1 results across all five repositories are available in the supplementary material accompanying this submission.